

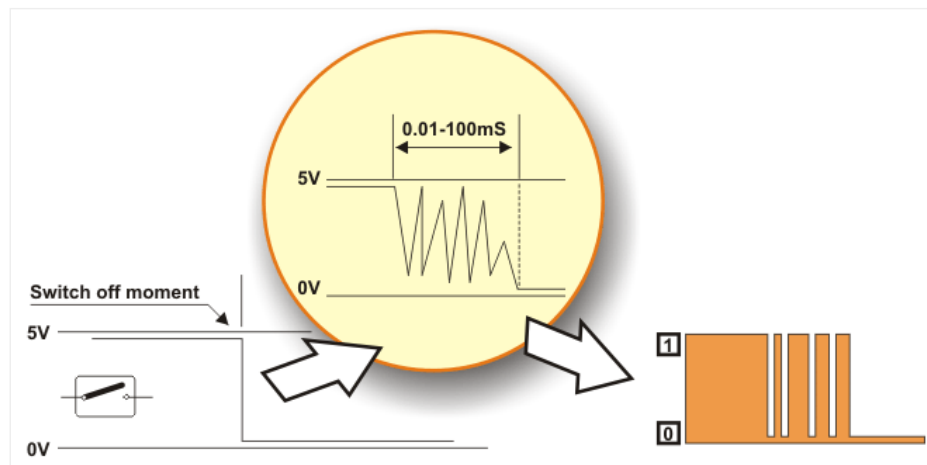
4.2 ADDITIONAL COMPONENTS

MIKROELEKTRONIKA

This section covers the most commonly used additional components in practice such as resistors, transistors, LED diodes, LED displays, LCD displays and RS232 communication circuits.

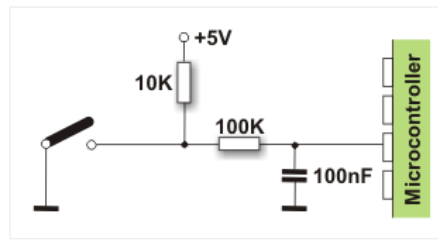
SWITCHES AND PUSH-BUTTONS

Switches and push-buttons are probably the simplest devices providing the simplest way of detecting the appearance of a voltage on a microcontroller input pin. Nevertheless, it is not as simple as it seems... The reason for it is a contact bounce.

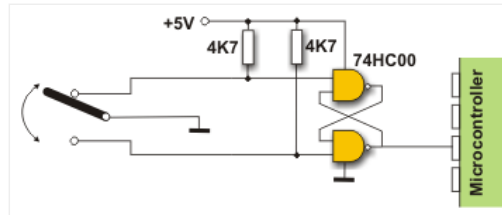


The contact bounce is a common problem with mechanical switches. When the contacts strike together, their momentum and elasticity act together to cause bounce. The result is a rapidly pulsed electrical current instead of a clean transition from zero to full current. It mostly occurs due to vibrations, slight rough spots and dirt between contacts. This effect is usually unnoticeable when using these components in everyday life because the bounce happens too fast to affect most equipment. However, it causes problems in some analog and logic circuits that respond fast enough to misinterpret on/off pulses as a data stream. Anyway, the whole process doesn't last long (a few micro or milliseconds), but long enough to be registered by the microcontroller. When only a push-button is used as a counter signal source, errors occur in almost 100% of cases!

This problem may be easily solved by connecting a simple RC circuit to suppress quick voltage changes. Since the bounce period is not defined, the values of components are not precisely determined. In most cases it is recommended to use the values as shown in figure below.



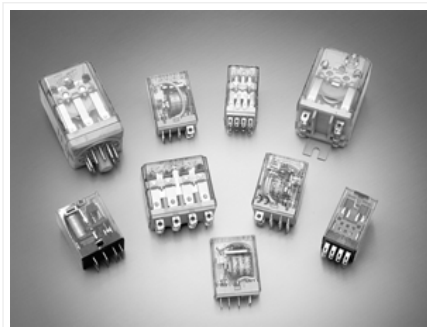
If complete stability is needed then radical measures should be taken. The output of the circuit, shown in figure below (RS flip-flop), will change its logic state only after detecting the first pulse triggered by a contact bounce. This solution is more expensive (SPDT switch), but the problem is definitely solved.



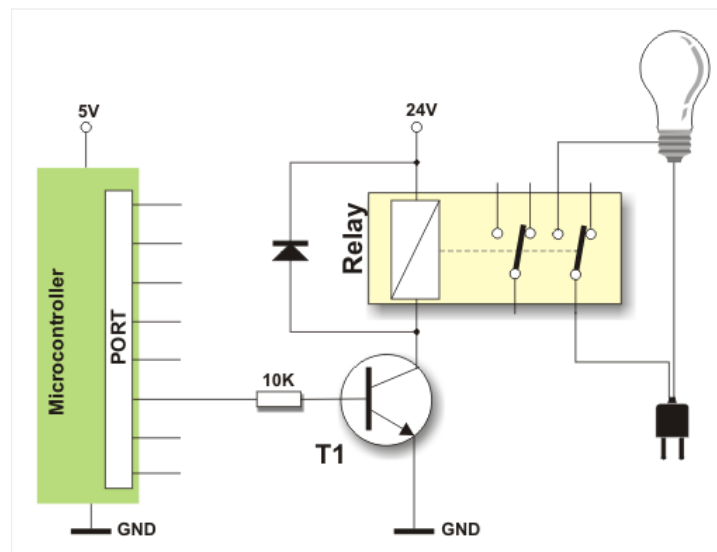
In addition to these hardware solutions, there is also a simple software solution. When the program tests the logic state of an input pin and detects a change, the check should be done one more time after a certain delay. If the program confirms the change, it means that a switch/push button has changed its position. The advantages of such solution are obvious: it is free of charge, effects of contact bounce are eliminated and it can be applied to the poorer quality contacts as well.

RELAY

A relay is an electrical switch that opens and closes under the control of another electrical circuit. It is therefore connected to output pins of the microcontroller and used to turn on/off high-power devices such as motors, transformers, heaters, bulbs, etc. These devices are almost always placed away from the board's sensitive components. There are various types of relays, but all of them operate in the same way. When current flows through the coil, the relay is operated by an electromagnet to open or close one or more sets of contacts. Similar to optocouplers, there is no galvanic connection (electrical contact) between input and output circuits. Relays usually demand both higher voltage and higher current to start operation, but there are also miniature ones that can be activated by low current directly obtained from a microcontroller pin.



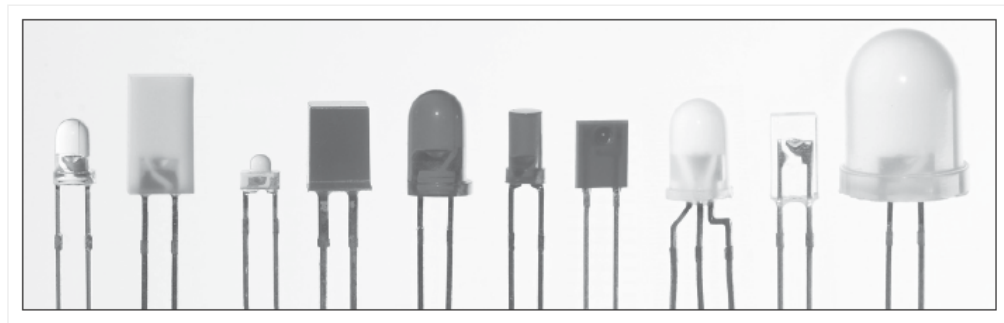
This figure below shows the most commonly used solution.



In order to prevent the appearance of high voltage self-induction, caused by a sudden stop of the current flow through the coil, an inverted polarized diode is connected in parallel to the coil. The purpose of this diode is to 'cut off' the voltage peak.

LED DIODES

You probably know all you need to know about LED diodes, but you should also think of the younger generations... Let's see, how to destroy an LED?! Well...Easily.



Quick Burning

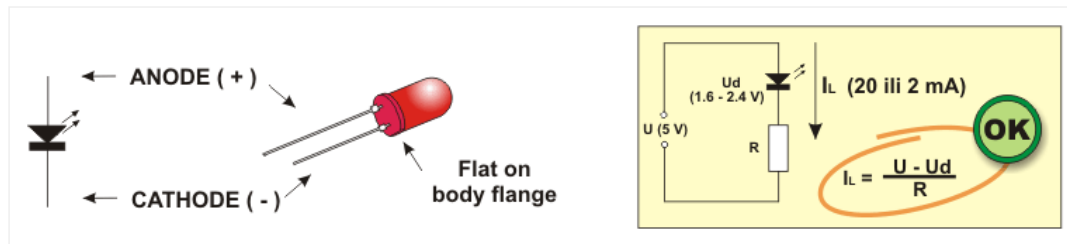
Like any other diode, LEDs have two ends- an anode and a cathode. Connect a diode properly to the power supply voltage and it will happily emit light. Turn the diode upside down and apply the same power supply voltage (even for a moment). It will not emit light – NEVER AGAIN!

Slow Burning

There is a nominal, i.e. maximum current limitation specified for every LED which must not be exceeded. If it happens, the diode will emit more intensive light, but just for a short period of time.

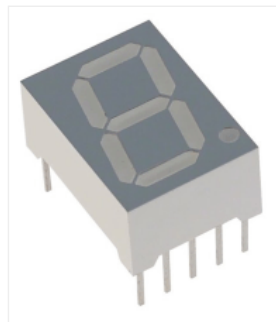
Something to Remember

Similarly, all you need to do is to discard a current limiting resistor shown below. Depending on the power supply voltage, the effects might be spectacular!



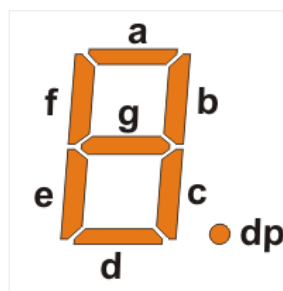
LED DISPLAY

Basically, an LED display is nothing more than several LEDs molded in the same plastic case. There are many types of displays and some of them are composed of several dozens built-in diodes which can display different symbols. Nevertheless, the most commonly used display is the 7-segment display. It is composed of 8 LEDs. Seven segments of a digit are arranged as a rectangle for symbol displaying, whereas the additional segment is used for the purpose of displaying decimal point. In order to simplify connection, anodes or cathodes of all diodes are connected to the common pin so that there are common anode displays and common cathode displays, respectively. Segments are marked with the letters from a to g, plus dp, as shown in figure below. When connecting, each diode is treated separately, which means that each must have its own current limiting resistor.



Here are a few important things that you should pay attention to when buying LED displays:

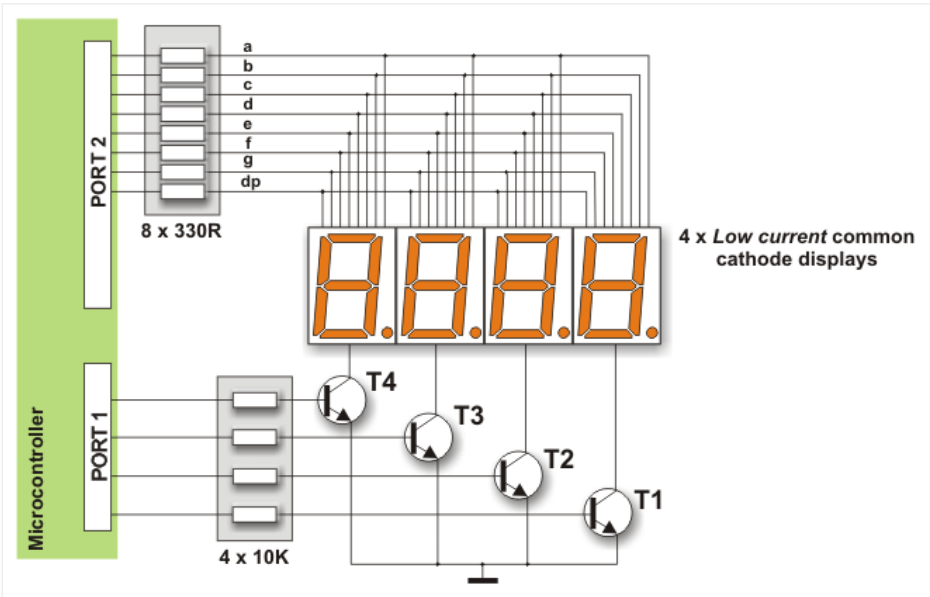
- As mentioned, depending on whether anodes or cathodes are connected to the common pin, there are common anode displays and common cathode displays. As for their appearance, there is no difference between these displays at all so it is recommended to check carefully prior to installing them which one is used.
- Each microcontroller pin has a maximum current limitation it can receive or give. Thus, if several displays are connected to the microcontroller it is recommended to use the so called *Low current* LEDs using only 2mA for the operation.
- Display segments are usually marked with the letters from a to g, but there is no fast rule indicating to which display pins they are connected. For this reason it is very important to check connecting prior to commencing writing a program or designing a device.



Displays connected to the microcontroller usually occupy a large number of valuable I/O pins, which can be a big problem especially when it is needed to display multi digit numbers. The problem is more than obvious if, for example, it is needed to display two 6-digit numbers (a simple calculation shows that 96 output pins are needed in this case). The solution to this problem is called MULTIPLEXING.

Here is how an optical illusion based on the same operating principle as a film camera is made. Only one digit at a time is active, but they change their on/off conditions so quickly making impression that all digits of a number

are simultaneously active.

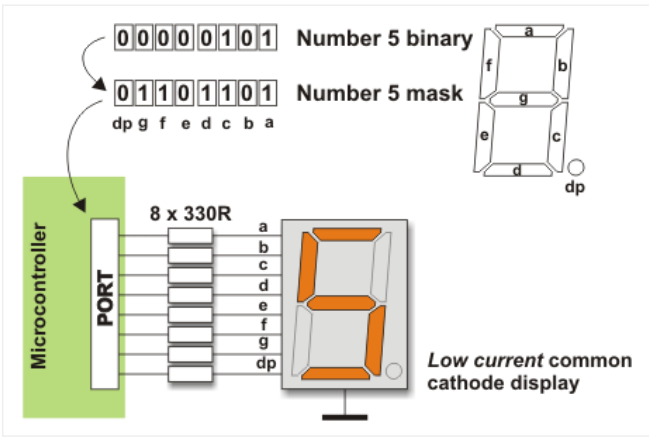


Here is an explanation on the figure above. First a byte representing units is applied on a microcontroller PORT2 and the transistor T1 is activated at the same time. After a while, the transistor T1 is turned off, a byte representing tens is applied on the PORT2 and the transistor T2 is activated. This process is being cyclicly repeated at high speed for all digits and corresponding transistors.

A disappointing fact which indicates that the microcontroller is just a kind of miniature computer designed to understand only the language of zeros and ones is fully expressed when displaying any digit. Namely, the microcontroller does not know what units, tens or hundreds are, nor what ten digits we are used to look like. For this reason, each number to be displayed must go through the following procedure:

First of all, a multi digit number must be split into units, tens etc. in a special subroutine. Then each of these digits must be stored in specific bytes. Digits get recognizable appearance by performing ‘masking’. In other words, the binary format of each digit is replaced by a different combination of bits using a simple subroutine. For example, the digit 8 (0000 1000) is replaced by the binary number 0111 1111 in order to activate all LEDs displaying the digit 8. The only diode remaining inactive here is reserved for the decimal point.

If a microcontroller port is connected to the display in such a way that bit 0 activates segment ‘a’, bit 1 activates segment ‘b’, bit 2 segment ‘c’ etc., then the table below shows the mask for each digit.



DIGITS TO DISPLAY	DISPLAY SEGMENTS							
	dp	a	b	c	d	e	f	g
0	0	1	1	1	1	1	1	0
1	0	0	1	1	0	0	0	0
2	0	1	1	0	1	1	0	1
3	0	1	1	1	1	0	0	1

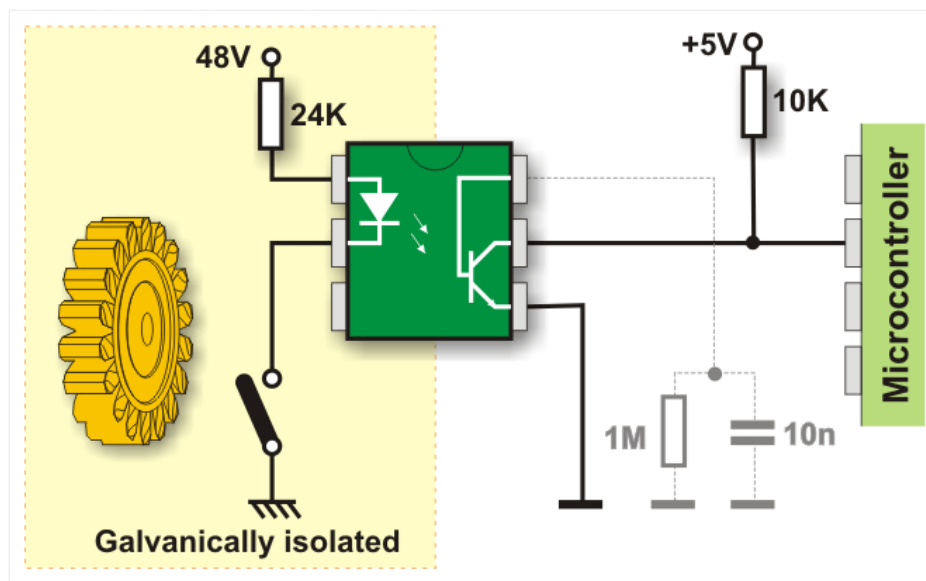
4	0	0	1	1	0	0	1	1
5	0	1	0	1	1	0	1	1
6	0	1	0	1	1	1	1	1
7	0	1	1	1	0	0	0	0
8	0	1	1	1	1	1	1	1
9	0	1	1	1	1	0	1	1

In addition to digits from 0 to 9, there are some letters- A, C, E, J, F, U, H, L, b, c, d, o, r, t, that can also be displayed by masking.

In the event that common anode displays are used, all ones contained in the previous table should be replaced by zeros and vice versa. Additionally, PNP transistors should be used as drivers.

OPTOCOUPLER

An optocoupler is a device commonly used to galvanically separate microcontroller electronics from any potentially dangerous current or voltage in its surroundings. Optocouplers usually have one, two or four light sources (LED diodes) on their input while on their output, opposite to diodes, there is the same number of elements sensitive to light (phototransistors, photo-thyristors or photo-triacs). The point is that an optocoupler uses a short optical transmission path to transfer a signal between the elements of circuit, while keeping them electrically isolated. This isolation makes sense only if diodes and photosensitive elements are separately powered. In this way, the microcontroller and expensive additional electronics are completely protected from high voltage and noises which are the most common cause of destroying, damaging or unstable operation of electronic devices in practice. The most frequently used optocouplers are those with phototransistors on their outputs. When it comes to the optocouplers with internal base-to-pin 6 connection (there are also optocouplers without it), the base can be left unconnected.

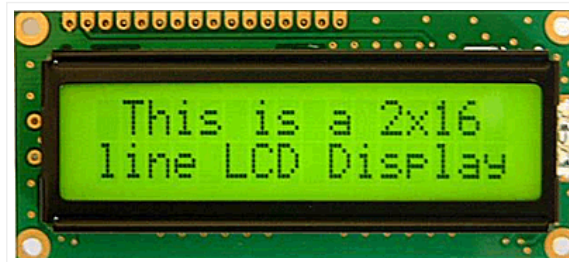


The R/C network represented by a broken line in the figure above denotes an optional connection which lessens the effects of noises by eliminating very short pulses.

LCD DISPLAY

This component is specifically manufactured to be used with microcontrollers, which means that it cannot be activated by standard IC circuits. It is used for displaying different messages on a miniature liquid crystal display. The model described here is for its low price and great capabilities most frequently used in practice. It is based on the HD44780 microcontroller (*Hitachi*) and can display messages in two lines with 16 characters each. It can display all the letters of alphabet, Greek letters, punctuation marks, mathematical symbols etc. It is also possible

to display symbols made up by the user. Other useful features include automatic message shift (left and right), cursor appearance, LED backlight etc.



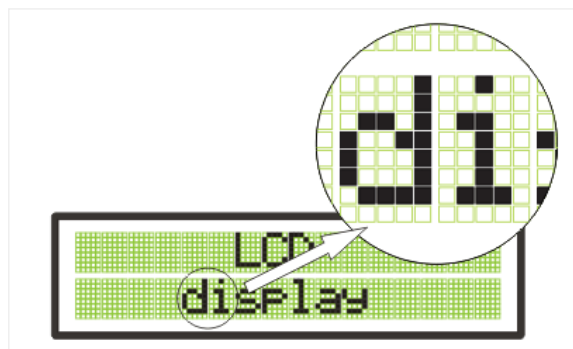
LCD Display Pins

Along one side of the small printed board of the LCD display there are pins that enable it to be connected to the microcontroller. There are in total of 14 pins marked with numbers (16 if there is a backlight). Their function is described in the table below:

FUNCTION	PIN NUMBER	NAME	LOGIC STATE	DESCRIPTION
Ground	1	Vss	–	0V
Power supply	2	Vdd	–	+5V
Contrast	3	Vee	–	0 – Vdd
Control of operating	4	RS	0 1	D0 – D7 are interpreted as commands D0 – D7 are interpreted as data
	5	R/W	0 1	Write data (from controller to LCD) Read data (from LCD to controller)
	6	E	0 1 From 1 to 0	Access to LCD disabled Normal operating Data/commands are transferred to LCD
Data / commands	7	D0	0/1	Bit 0 LSB
	8	D1	0/1	Bit 1
	9	D2	0/1	Bit 2
	10	D3	0/1	Bit 3
	11	D4	0/1	Bit 4
	12	D5	0/1	Bit 5
	13	D6	0/1	Bit 6
	14	D7	0/1	Bit 7 MSB

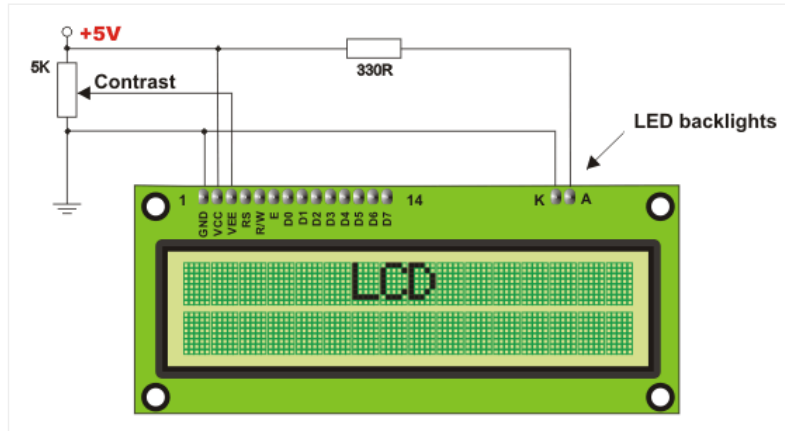
LCD Screen

An LCD screen can display two lines with 16 characters each. Every character consists of 5×8 or 5×11 dot matrix. This book covers a 5×8 character display which is most commonly used.



Display contrast depends on the power supply voltage and whether messages are displayed in one or two lines. For this reason, varying voltage 0-Vdd is applied to the pin marked as Vee. A trimmer potentiometer is usually

used for this purpose. Some of the LCD displays have built-in backlight (blue or green LEDs). When used during operation, a current limiting resistor should be serially connected to one of the pins for backlight power supply (similar to LED diodes).



If there are no characters displayed or if all of them are dimmed when the display is switched on, the first thing that should be done is to check the potentiometer for contrast adjustment. Is it properly adjusted? The same applies if the mode of operation has been changed (writing in one or two lines).

LCD Memory

LCD display contains three memory blocks:

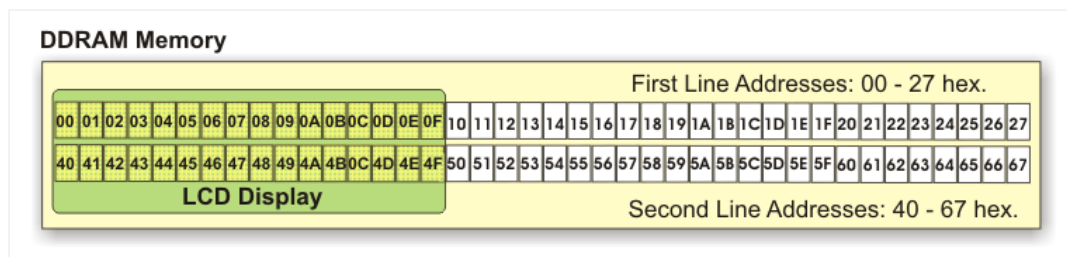
- DDRAM Display Data RAM;
- CGRAM Character Generator RAM; and
- CGROM Character Generator ROM.

DDRAM Memory

DDRAM memory is used for storing characters to be displayed. The size of this memory is capable of storing 80 characters. Some memory locations are directly connected to the characters on display.

Everything works quite simply: it is enough to configure the display to increment addresses automatically (shift right) and set the starting address for the message to be displayed (for example 00 hex).

Afterwards, all characters sent through lines D0-D7 will be displayed in the message format we are used to- from left to right. In this case, displaying starts from the first field of the first line because the initial address is 00 hex. If more than 16 characters are sent, then all of them will be memorized, but only the first sixteen characters will be visible. In order to display the rest of them, the shift command should be used. Virtually, everything looks as if the LCD display is a window which *shifts* left-right over memory locations containing different characters. In reality, this is how the effect of the message shifting over the screen has been created.



If the cursor is on, it appears at the currently addressed location. In other words, when a character appears at the cursor position, it will automatically move to the next addressed location.

This is a sort of RAM memory so that data can be written to and read from it, but its content is irretrievably lost when the power goes off.

CGROM Memory

CGROM memory contains a standard character map with all characters that can be displayed on the screen. Each character is assigned to one memory location:

		4 higher bits of address															
		0000	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101	1110	1111
4 lower bits of address	xxxx0000	CG RAM (1)			0	0	P	`	P				-	9	3	α	p
	xxxx0001	(2)			!	1	A	Q	a	q			.	7	4	ä	q
	xxxx0010	(3)			"	2	B	R	b	r			'	イ	ツ	×	ρ
	xxxx0011	(4)			#	3	C	S	c	s			」	ウ	〒	ε	ω
	xxxx0100	(5)			\$	4	D	T	d	t			、	エ	ト	μ	Ω
	xxxx0101	(6)			%	5	E	U	e	u			・	オ	ナ	1	ü
	xxxx0110	(7)			&	6	F	V	f	v			ヲ	カ	ニ	ヨ	ρ
	xxxx0111	(8)			'	7	G	W	g	w			フ	キ	ヲ	ウ	g
	xxxx1000	(1)			(	8	H	X	h	x			イ	ク	ネ	リ	フ
	xxxx1001	(2)			)	9	I	Y	i	y			ッ	ケ	ル	ニ	4
	xxxx1010	(3)			*	:	J	Z	j	z			エ	コ	ン	レ	j
	xxxx1011	(4)			+	;	K	[	k	{			オ	サ	ヒ	ロ	*

The addresses of CGROM memory locations match the characters of ASCII. If the program being currently executed encounters a command 'send character P to port' then the binary value 0101 0000 appears on the port. This value is the ASCII equivalent to the character P. It is then written to an LCD, which results in displaying the symbol from the 0101 0000 location of CGROM. In other words, the character 'P' is displayed. This applies to all letters of alphabet (capitals and small), but not to numbers. As seen on the previous map, addresses of all digits are pushed forward by 48 relative to their values (digit 0 address is 48, digit 1 address is 49, digit 2 address is 50 etc.). Accordingly, in order to display digits correctly it is necessary to add the decimal number 48 to each of them prior to being sent to an LCD.

What is ASCII? From their inception till today, computers can recognize only numbers, but not letters. It means that all data a computer swaps with a peripheral device has a binary format even though the same is recognized by the man as letters (the keyboard is an excellent example). In other words, every character matches a unique combination of zeroes and ones. ASCII is character encoding based on the English alphabet. ASCII code specifies a correspondence between standard character symbols and their numerical equivalents.

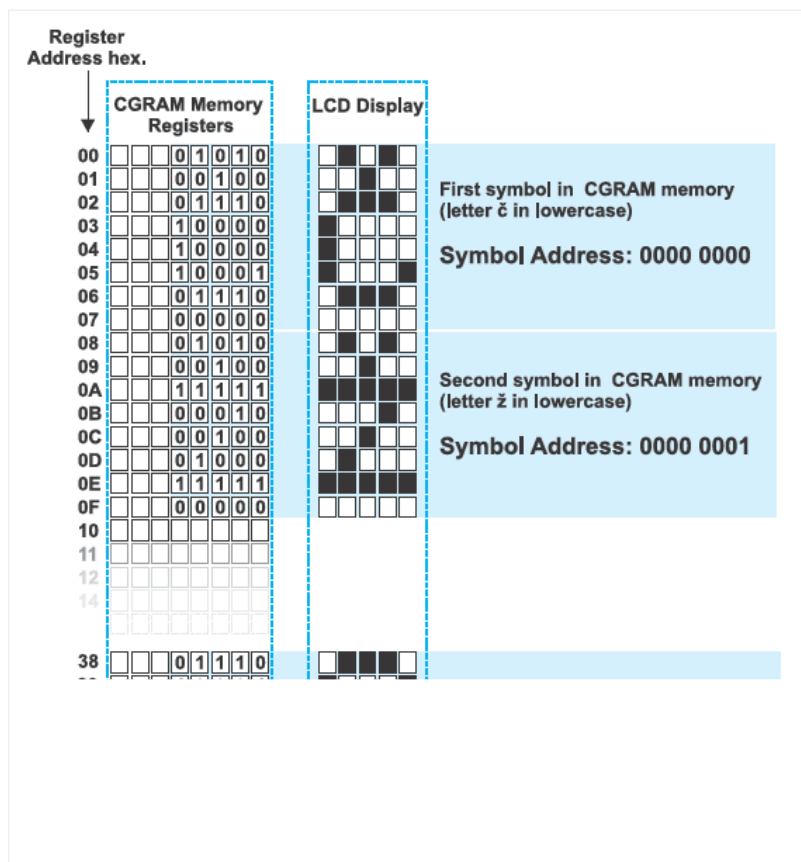
ASCII Hex Symbol			ASCII Hex Symbol			ASCII Hex Symbol			ASCII Hex Symbol		
0	0	NUL	16	10	DLE	32	20	(space)	48	30	0
1	1	SOH	17	11	DC1	33	21	!	49	31	1
2	2	STX	18	12	DC2	34	22	"	50	32	2
3	3	ETX	19	13	DC3	35	23	#	51	33	3
4	4	EOT	20	14	DC4	36	24	\$	52	34	4
5	5	ENQ	21	15	NAK	37	25	%	53	35	5
6	6	ACK	22	16	SYN	38	26	&	54	36	6
7	7	BEL	23	17	ETB	39	27	'	55	37	7
8	8	BS	24	18	CAN	40	28	(	56	38	8
9	9	TAB	25	19	EM	41	29	)	57	39	9
10	A	LF	26	1A	SUB	42	2A	*	58	3A	:
11	B	VT	27	1B	ESC	43	2B	+	59	3B	;
12	C	FF	28	1C	FS	44	2C	,	60	3C	<
13	D	CR	29	1D	GS	45	2D	-	61	3D	=
14	E	SO	30	1E	RS	46	2E	.	62	3E	>
15	F	SI	31	1F	US	47	2F	/	63	3F	?

ASCII Hex Symbol			ASCII Hex Symbol			ASCII Hex Symbol			ASCII Hex Symbol		
64	40	@	80	50	P	96	60	`	112	70	p
65	41	A	81	51	Q	97	61	a	113	71	q
66	42	B	82	52	R	98	62	b	114	72	r
67	43	C	83	53	S	99	63	c	115	73	s
68	44	D	84	54	T	100	64	d	116	74	t
69	45	E	85	55	U	101	65	e	117	75	u
70	46	F	86	56	V	102	66	f	118	76	v
71	47	G	87	57	W	103	67	g	119	77	w
72	48	H	88	58	X	104	68	h	120	78	x
73	49	I	89	59	Y	105	69	i	121	79	y
74	4A	J	90	5A	Z	106	6A	j	122	7A	z
75	4B	K	91	5B	[	107	6B	k	123	7B	{
76	4C	L	92	5C	\	108	6C	l	124	7C	
77	4D	M	93	5D	]	109	6D	m	125	7D	}
78	4E	N	94	5E	^	110	6E	n	126	7E	~
79	4F	O	95	5F	_	111	6F	o	127	7F	

CGRAM Memory

Apart from standard characters, the LCD display can also display symbols defined by the user itself. It can be any symbol in the size of 5×8 pixels. RAM memory called CGRAM in the size of 64 bytes enables it.

Memory registers are 8 bits wide, but only 5 lower bits are used. Logic one (1) in every register represents a dimmed dot, while 8 locations grouped together represent one character. It is best illustrated in figure below:



Symbols are usually defined at the beginnig of the program by simple writing zeros and ones to registers of CGRAM memory so that they form desired shapes. In order to display them it is sufficient to specify their address. Pay attention to the first coloumn in the CGROM map of characters. It doesn't contain RAM memory addresses,

but symbols being discussed here. In this example, 'display 0' means – display 'č', 'display 1' means – display 'ž' etc.

LCD Basic Commands

All data transferred to an LCD through the outputs D0-D7 will be interpreted as a command or a data, which depends on the RS pin logic state:

- **RS = 1** – Bits D0 – D7 are addresses of the characters to be displayed. LCD processor addresses one character from the character map and displays it. The DDRAM address specifies location on which the character is to be displayed. This address is defined prior to transferring character or the address of the previously transferred character is automatically incremented.
- **RS = 0** – Bits D0 – D7 are commands for setting the display mode.

Here is a list of commands recognized by the LCD:

COMMAND	RS	RW	D7	D6	D5	D4	D3	D2	D1	D0	EXECUTION TIME
Clear display	0	0	0	0	0	0	0	0	0	1	1.64ms
Cursor home	0	0	0	0	0	0	0	0	1	x	1.64ms
Entry mode set	0	0	0	0	0	0	0	1	I/D	S	40uS
Display on/off control	0	0	0	0	0	0	1	D	U	B	40uS
Cursor/Display Shift	0	0	0	0	0	1	D/C	R/L	x	x	40uS
Function set	0	0	0	0	1	DL	N	F	x	x	40uS
Set CGRAM address	0	0	0	1	CGRAM address						40uS
Set DDRAM address	0	0	1	DDRAM address							40uS
Read "BUSY" flag (BF)	0	1	BF	DDRAM address							–
Write to CGRAM or DDRAM	1	0	D7	D6	D5	D4	D3	D2	D1	D0	40uS
Read from CGRAM or DDRAM	1	1	D7	D6	D5	D4	D3	D2	D1	D0	40uS

```

1 I/D 1 = Increment (by 1)      R/L 1 = Shift right
2   0 = Decrement (by 1)      0 = Shift left
3
4 S 1 = Display shift on      DL 1 = 8-bit interface
5   0 = Display shift off    0 = 4-bit interface
6
7 D 1 = Display on           N 1 = Display in two lines
8   0 = Display off         0 = Display in one line
9
10 U 1 = Cursor on           F 1 = Character format 5x10 dots
11   0 = Cursor off         0 = Character format 5x7 dots
12
13 B 1 = Cursor blink on    D/C 1 = Display shift
14   0 = Cursor blink off   0 = Cursor shift

```

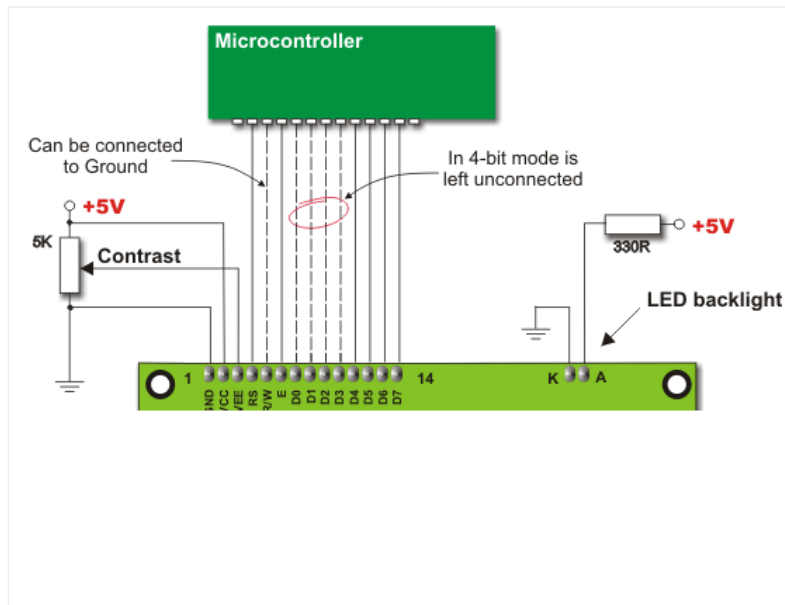
WHAT IS THE BUSY FLAG?

Compared to the microcontroller, the LCD is an extremely slow component. For this reason, it was necessary to provide a signal which would, upon command execution, indicate that the display is ready for the next piece of data. That signal, called the *busy flag*, can be read from the line D7. The display is ready to receive new data when the voltage on this line is 0V (BF=0).

LCD Connecting

Depending on how many lines are used for connecting an LCD to the microcontroller, there are 8-bit and 4-bit LCD modes. The appropriate mode is selected at the beginning of the operation in the process called 'initialization'. The 8-bit LCD mode uses outputs D0- D7 to transfer data as explained on the previous page.

The main purpose of the 4-bit LCD mode is to save valuable I/O pins of the microcontroller. Only 4 higher bits (D4-D7) are used for communication, while others may be left unconnected. Each piece of data is sent to the LCD in two steps- four higher bits are sent first (normally through the lines D4-D7), then four lower bits. Initialization enables the LCD to link and interpret received bits correctly.



Data is rarely read from the LCD (it is mainly transferred from the microcontroller to the LCD) so it is often possible to save an extra I/O pin by simply connecting the R/W pin to the Ground. Such a saving has its price. Messages will be normally displayed, but it will not be possible to read the busy flag since it is not possible to read the display either. Fortunately, there is a simple solution. After sending a character or a command it is important to give the LCD enough time to do its job. Owing to the fact that the execution of a command may last for approximately 1.64mS, it will be sufficient to wait about 2mS for the LCD.

LCD Initialization

The LCD is automatically cleared when powered up. It lasts for approximately 15mS. After this, it is ready for operation. The mode of operation is set by default, which means that:

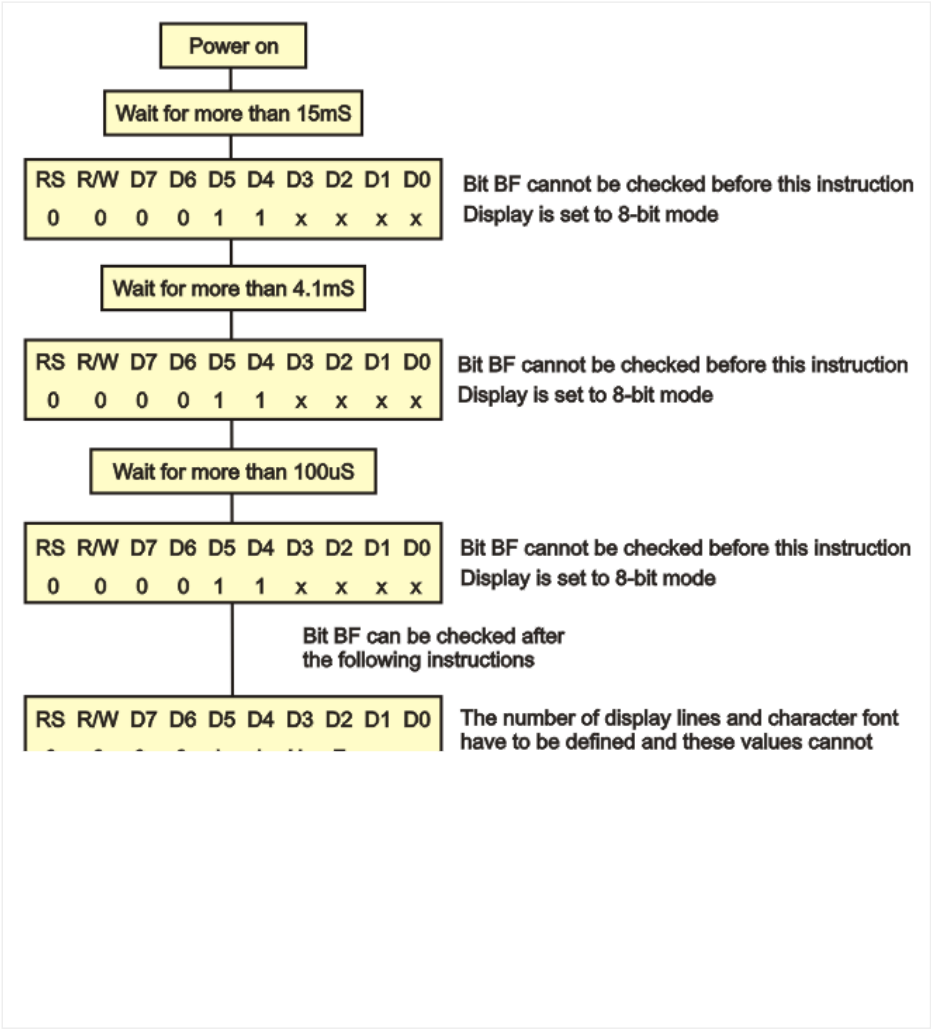
1. Display is cleared.
2. Mode
 - DL** = 1 – Communication through 8-bit interface
 - N** = 0 – Messages are displayed in one line
 - F** = 0 – Character font 5 x 8 dots
3. Display/Cursor on/off
 - D** = 0 – Display off
 - U** = 0 – Cursor off
 - B** = 0 – Cursor blink off
4. Character entry
 - ID** = 1 Displayed addresses are automatically incremented by 1
 - S** = 0 Display shift off

Automatic reset mostly occurs without any problems. Mostly, but not always! If for any reason the power supply voltage doesn't reach full value within 10mS, the display will start to perform completely unpredictably. If the

voltage unit is not able to meet that condition or if it is needed to provide completely safe operation, the process of initialization is applied. Initialization, among other things, causes a new reset by enabling the display to operate normally.

There are two initialization algorithms. Which one is to be performed depends on whether connecting to the microcontroller is through 4- or 8-bit interface. In both cases, all that's left to do after initialization is to specify basic commands and of course – to display messages.

Refer to figure below for the procedure in 8-bit initialization:



It is not a mistake! In this algorithm, the same value is transferred three times in a row.

The procedure in 4-bit initialization is as follows:



Let's do it in mikroC...

```

1 /* In mikroC for PIC, it is sufficient to write only one function to perform all
2    described operations for LCD initialization. */
3
4 ...
5 Lcd_Init(); // Initialize LCD
6 ...

```



4.2 Additional Components by MikroElektronika is licensed under a Creative Commons Attribution 4.0 International License, except where otherwise noted.

